Grounded Document Search

Project overview

Retrieve relevant passages from a tiny knowledge base with TF-IDF and return extractive answers with source citations.

This guide explains the delivered implementation, how to run it, what each component does, the algorithms, and the limits of the demonstration.

Prepared for AI Product Reviewer / Beta-Testing internship interviews. Synthetic data. Local demo. AI-assisted starter; personalize and understand before presenting.

Grounded Document Search

Problem and demo

Support teams need answers that can be traced to source documents.

Search for refund, shipping or API keys and inspect the returned passage and source.

Ask about volcanoes to see an explicit no-evidence result.

This app returns existing text instead of generating an answer, so it is a transparent retrieval foundation for a future RAG assistant.

Grounded Document Search

Setup and repository map

Prerequisite: Python 3.10 or newer.

No pip install or API key is required.

Extract this ZIP, open a terminal in its project folder and run python server.py.

Open http://127.0.0.1:8000 in your browser.

If that port is busy, run python server.py --port 8001 and open port 8001 instead.

On Windows, py can replace python.

Stop with Ctrl+C.

Run python -m unittest discover -v to execute tests.

logic.py holds domain logic; server.py handles HTTP and local storage; index.html contains the browser UI; sample_data.json holds synthetic fixtures; test_logic.py checks domain behavior; test_http.py checks the HTTP boundary.

requirements.txt documents the standard-library-only runtime.

The guide-generation tools are not runtime dependencies.

Grounded Document Search

Architecture and technology choices

Browser form -> fetch POST /api/run -> Python HTTP handler -> domain run(payload, records) -> JSON result -> safe DOM tables.

GET / serves the app, GET /api/health returns a health response, and unknown paths return 404.

Python provides a portable runtime; http.server provides a minimal HTTP boundary; JSON keeps fixtures inspectable; HTML and CSS provide a responsive UI; JavaScript fetch connects the UI and backend; unittest provides repeatable checks.

Browser output uses textContent rather than HTML injection.

There is no framework, build system, external CDN, telemetry upload or database dependency.

Grounded Document Search

Retrieval explained

Tokenization uses lowercase alphanumeric words.

Each passage is represented by term counts multiplied by smoothed inverse document frequency: count * (1 + log((1 + N)/(1 + document frequency))).

The query uses the same weighting.

Cosine similarity compares vector direction, preventing long passages from winning merely by length.

The top three positive matches are returned; the first becomes the extractive answer.

It uses term counts, without length-normalized term frequency or stemming.

Grounded Document Search

Code walkthrough

logic.py exposes tokens() and run().

run() validates the query, builds document counters, calculates weights, ranks passages and returns source labels plus scores.

sample_data.json stores one passage per document.

The source names are citation identifiers, not links to additional files.

Scores describe lexical similarity, not confidence or factual accuracy.

The UI shows raw structured evidence so an interviewer can inspect retrieval decisions.

Limitations and AI extension

Synonyms such as money back versus refund can fail; common words can produce irrelevant matches.

A positive score is not a reliable relevance threshold.

The corpus is tiny and synthetic.

Add chunking, a relevance benchmark, held-out queries, calibrated thresholds and access control.

For semantic search replace vectors with embeddings and retain source metadata.

If adding a generator, send only retrieved context, request citations, test unsupported questions, and treat document text as data rather than instructions.

Interview discussion and next steps

Walk through query -> tokenize -> vectorize -> rank -> cite.

Explain why citation presence does not prove an answer is correct and why an extractive response reduces but does not eliminate misleading answers.

Add your own public documents with provenance, evaluate recall at k and mean reciprocal rank, and compare lexical and embedding search.

Test malicious document instructions before connecting a language model.

API, errors and operational limits

POST /api/run accepts a JSON object with Content-Type application/json.

The form fields below correspond to payload keys.

Successful calls return HTTP 200 and JSON; validation or malformed JSON returns 400; unknown paths return 404.

Bodies must be between 1 and 32768 bytes.

Unexpected local data or storage errors return 500 with a generic message.

The server binds to 127.0.0.1 and is intended for a trusted local demo.

Python http.server is not a production server.

Authentication, TLS, concurrent workers, rate limits and schema validation must be designed before deployment.

Do not expose this process directly to the internet.

Example request payload

```
{
  "query": "refund policy"
}
```

Testing, GitHub and ownership

Run the logic and HTTP tests before changing sample data, since expected values describe the shipped fixture.

The HTTP tests start a temporary local server and verify health, successful workflow, malformed JSON, non-object payloads and unknown paths.

This is a useful smoke suite, not comprehensive security or browser coverage.

Create a GitHub repository, copy the extracted project files into it, then run git init, git add ., git commit -m "Initial portfolio demo", git branch -M main, git remote add origin YOUR_REPOSITORY_URL, and git push -u origin main.

Replace the URL with your own empty repository.

Add a screenshot and a short video after running it.

This is an AI-assisted starter project; describe your own review, changes and learning accurately.

Do not claim customer usage, production deployment or measured model improvements from these synthetic examples.